OrgaLego

Un simulateur 2D pour la conception et le diagnostic de structures organisationnelles

Spécification technique et fonctionnelle du projet

Document de conception

16 mai 2026

Résumé

OrgaLego est un simulateur 2D interactif permettant de concevoir et diagnostiquer des structures organisationnelles à partir d'une bibliothèque de blocs préfabriqués issus des grandes écoles de pensée (Mintzberg, Fayol, Weber, etc.). L'organisation est modélisée comme un graphe hiérarchique à trois niveaux (macro, méso, micro), affichés successivement par navigation : l'utilisateur clique sur un nœud pour descendre dans la structure et explorer le niveau correspondant. Le diagnostic s'appuie sur trois algorithmes de théorie des graphes : détection de communautés (Louvain), centralité d'intermédiarité, et matching d'archétypes théoriques. L'architecture combine un *backend* Python (NetworkX, FastAPI) et un *frontend* en React communiquant par WebSocket.

Table des matières

1 Problématique : pourquoi cet outil doit exister **3**
- 1.1 Le coût caché des restructurations . . . 3
- 1.2 La densité du corpus théorique . . . 3
- 1.3 Les limites des outils existants . . . 3
- 1.4 Le positionnement d'OrgaLego . . . 4

2 Modélisation conceptuelle : graphes hiérarchiques à trois niveaux **4**
- 2.1 Principe général . . . 4
- 2.2 Attributs portés par les nœuds et les arêtes . . . 4

3 Étude de cas : la société fictive TechNova **4**
- 3.1 Niveau 1 — Vue macro : architecture organisationnelle . . . 4
- 3.2 Niveau 2 — Vue méso : équipes au sein de la R&D . . . 4
- 3.3 Niveau 3 — Vue micro : individus au sein d'une équipe . . . 5
- 3.4 Vue intégrée : navigation entre niveaux par clic . . . 5

4 Algorithmes d'optimisation mis en œuvre **5**
- 4.1 Détection de silos par modularité (algorithme de Louvain) . . . 6
- 4.2 Détection de goulots par centralité d'intermédiarité . . . 7
- 4.3 Identification des configurations théoriques (matching d'archétypes) . . . 7
- 4.4 Algorithmes prévus pour les versions ultérieures . . . 7

5 Architecture technique **8**
5.1 Vue d'ensemble 8
5.2 Structure du projet et organisation du code 8
5.3 Backend Python : bibliothèques et rôle de chacune 10
5.4 Frontend : choix techniques 11
5.5 Communication backend ↔ frontend 11

6 Synthèse du scénario TechNova **12**

7 Conclusion et perspectives **13**

1 Problématique : pourquoi cet outil doit exister

1.1 Le coût caché des restructurations

La restructuration d'une entreprise est l'une des opérations les plus coûteuses et risquées qu'elle puisse entreprendre. Trois ordres de coûts s'y cumulent : **financiers** (honoraires de conseil souvent supérieurs à plusieurs centaines de milliers d'euros), **humains** (perte de productivité de 6 à 18 mois, démotivation, départs, perte de savoir-faire informel) et **d'opportunité** (mobilisation du management au détriment du business). Conséquence : les restructurations sont rares (typiquement tous les 5 à 10 ans), créant un paradoxe — la décision la plus structurante pour la performance est aussi celle pour laquelle les dirigeants ont la moins d'expérience.

1.2 La densité du corpus théorique

Le second obstacle est cognitif. La théorie des organisations s'est construite sur plus d'un siècle et compte une douzaine d'écoles, chacune avec ses concepts, auteurs et prescriptions :

École	**Auteurs**	**Apport principal**
Organisation scientifique du travail	Taylor	Division verticale et horizontale du travail, standardisation
Administrative	Fayol	Principes de direction, unité de commandement, span of control
Bureaucratique	Weber	Formalisation des règles, hiérarchie rationnelle-légale
Relations humaines	Mayo, Maslow	Importance des facteurs psycho-sociaux, motivation
Sociotechnique	Trist, Emery	Couplage entre système technique et système social
Contingence	Lawrence & Lorsch	Pas de modèle unique : la structure dépend de l'environnement
Configurationnelle	Mintzberg	5 à 7 configurations types (structure simple, bureaucratie mécaniste, professionnelle, divisionnelle, adhocratie, etc.)
Analyse stratégique	Crozier, Friedberg	Jeux de pouvoir et zones d'incertitude

TABLE 1 – Aperçu des principales écoles de pensée organisationnelle

Constat opérationnel : aucun dirigeant ou consultant ne peut tenir simultanément en tête la totalité de ce corpus pour l'appliquer de manière cohérente. Les décisions sont prises sur quelques principes intuitivement maîtrisés, au risque de produire des structures théoriquement incohérentes (par exemple un bloc bureaucratique wébérien greffé sur une adhocratie de Mintzberg, sans mécanisme de coordination compatible).

1.3 Les limites des outils existants

Des outils d'*Organizational Design Software* existent (OrgVue, Nakisa, Ingentis, Polinode, OrgWeaver) mais présentent trois limites :

1. **Data-driven, pas theory-driven** : ils analysent les données HR (Slack/Teams, reporting) sans s'appuyer explicitement sur les écoles théoriques.
2. **Orientés grandes entreprises** : tarification supérieure à 30k EUR/an, intégrations Workday/SAP requises, inaccessibles aux PME et à l'enseignement.
3. **Pédagogie limitée** : organigrammes plats sans représentation explicite des niveaux d'abstraction (division / équipe / individu) ni des archétypes théoriques associés.

1.4 Le positionnement d'OrgaLego

Proposition de valeur : OrgaLego est un simulateur 2D interactif qui permet aux étudiants, formateurs, consultants juniors et dirigeants de PME d'**apprendre** et de **tester** les théories d'organisation en assemblant des structures comme des Lego, et en observant en temps réel leurs forces et faiblesses. L'utilisateur navigue par clic entre les trois niveaux d'abstraction (division → équipe → individu). L'outil rend opérationnel un corpus théorique aujourd'hui réservé aux experts.

2 Modélisation conceptuelle : graphes hiérarchiques à trois niveaux

2.1 Principe général

Une organisation est représentée comme un **graphe hiérarchique multi-niveaux** composé de trois sous-graphes empilés et reliés par des arêtes inter-niveaux dites de *membership*. Cette décomposition est cohérente avec celle de Mintzberg (sommet stratégique, ligne hiérarchique, centre opérationnel) tout en restant manipulable par les algorithmes standards de théorie des graphes.

Formellement :

$$\mathcal{G} = (G_1, G_2, G_3, E_{1\to2}, E_{2\to3})$$

où G_1, G_2, G_3 sont respectivement les graphes **macro** (divisions, BU), **méso** (équipes, comités) et **micro** (individus), et $E_{1\to2}, E_{2\to3}$ sont les arêtes de rattachement pondérées (un individu peut appartenir à plusieurs équipes avec un % de temps alloué).

2.2 Attributs portés par les nœuds et les arêtes

Les **nœuds** portent : identifiant, libellé, type, capacité, coût, performance, charge, compétences. Les **arêtes intra-niveau** : type (hiérarchique, fonctionnelle, collaboration, informelle), fréquence, latence, criticité. Les **arêtes inter-niveau** : pondération, nature du rattachement (principal/secondaire, formel/informel).

3 Étude de cas : la société fictive TechNova

Pour illustrer concrètement la modélisation et les algorithmes, nous considérons **TechNova**, une scale-up européenne du secteur logiciel comptant 150 collaborateurs. L'entreprise rencontre des difficultés de coordination entre la R&D et le commercial, et envisage une restructuration.

3.1 Niveau 1 — Vue macro : architecture organisationnelle

Le niveau macro représente les grandes divisions de TechNova et les flux structurels entre elles. La figure 1 présente l'état initial.

Diagnostic macro : structure fonctionnelle pure. L'absence de lien direct R&D–Commercial est un *anti-pattern* classique : les besoins clients transitent par le Produit, créant un délai de réactivité préjudiciable.

3.2 Niveau 2 — Vue méso : équipes au sein de la R&D

En "zoomant" sur le nœud R&D du graphe précédent, on découvre le sous-graphe des équipes qui le compose (figure 2).

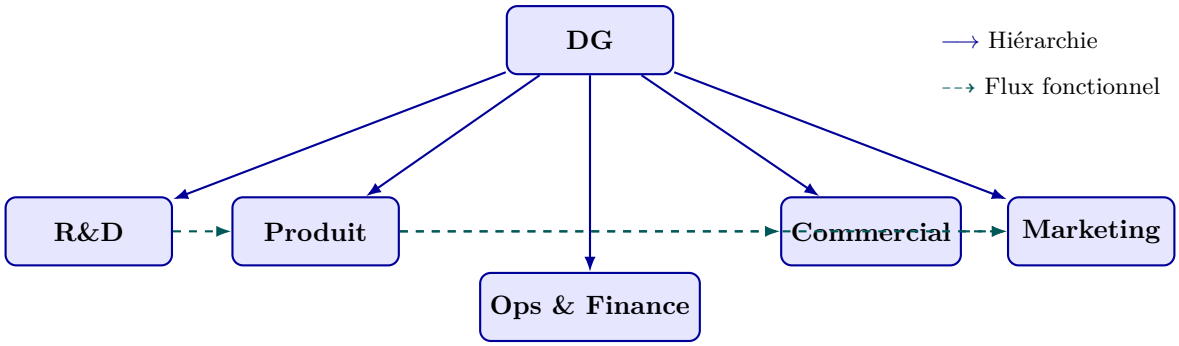


FIGURE 1 – Graphe macro de TechNova (G_1) — état initial. Structure fonctionnelle classique.

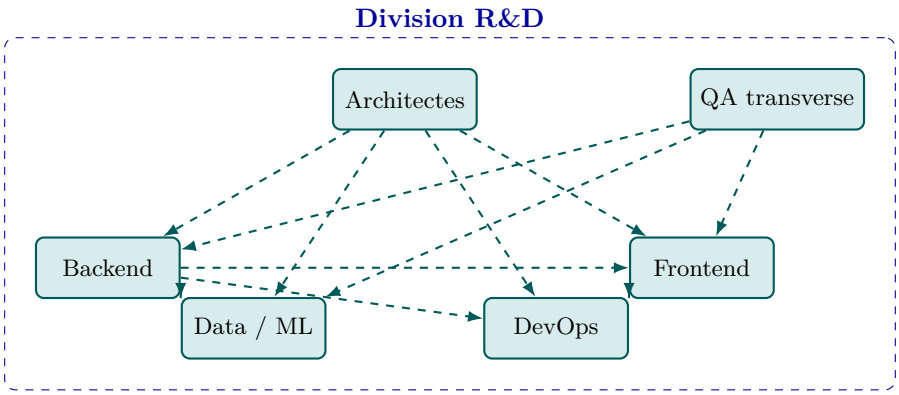


FIGURE 2 – Graphe méso de TechNova (G_2) — zoom sur la division R&D. L'équipe QA est transverse et intervient dans plusieurs équipes.

Diagnostic méso : l'équipe Architectes joue un rôle de coordination centrale. L'équipe QA est un *nœud à forte centralité d'intermédiarité* : son indisponibilité bloquerait simultanément plusieurs équipes.

3.3 Niveau 3 — Vue micro : individus au sein d'une équipe

En zoomant à nouveau sur l'équipe Backend, on découvre les individus et leurs relations de collaboration réelles (figure 3).

Diagnostic micro : Alex est un *leader informel* dont la centralité réelle excède largement son rôle formel — son départ serait un *single point of failure*. L'écart entre l'organigramme formel (étoile) et le réseau réel (maillage) révèle un fonctionnement par ajustement mutuel proche d'une **adhocratie**, alors que le reste de la R&D relève plutôt d'une **bureaucratie professionnelle** : cette incohérence inter-niveaux est un signal de tension.

3.4 Vue intégrée : navigation entre niveaux par clic

La figure 4 schématise la structure logique du graphe hiérarchique : les trois niveaux (macro, méso, micro) sont reliés par les arêtes verticales de *membership*. Dans l'application 2D, l'utilisateur ne voit qu'**un seul niveau à la fois** : un clic sur un nœud du niveau supérieur ouvre la vue du sous-graphe correspondant au niveau inférieur. Un fil d'Ariane permet de remonter.

4 Algorithmes d'optimisation mis en œuvre

OrgaLego s'appuie pour son MVP sur **trois algorithmes éprouvés** de théorie des graphes, chacun adressant une question organisationnelle précise et complémentaire des deux autres.

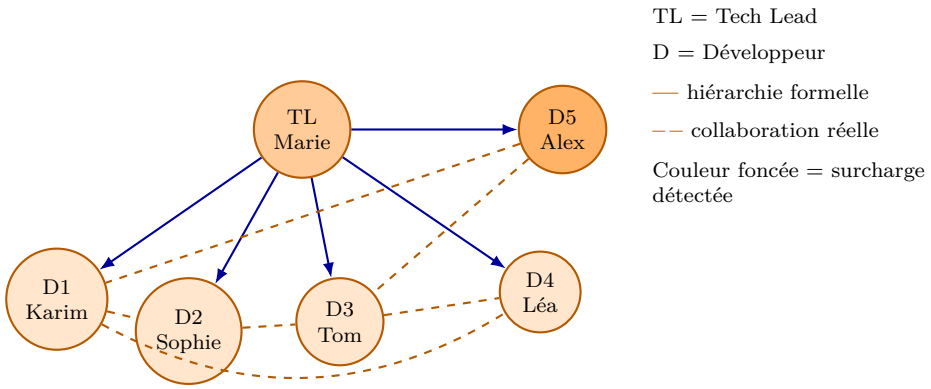


FIGURE 3 – Graphe micro de TechNova (G_3) — zoom sur l'équipe Backend. Alex (D5) apparaît surchargé : il est sollicité par plusieurs membres en dehors de la chaîne hiérarchique formelle.

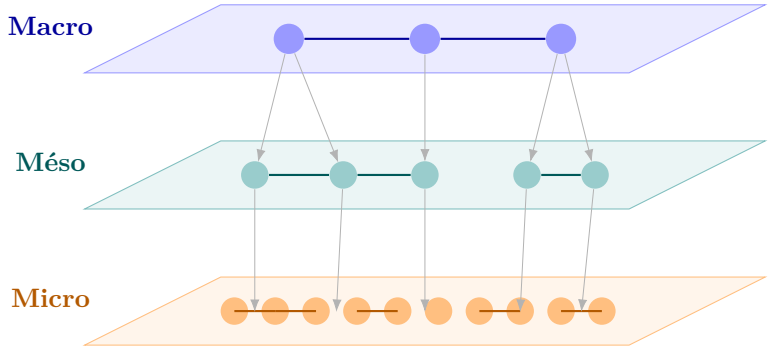


FIGURE 4 – Structure logique du graphe hiérarchique à trois niveaux, reliés par les arêtes de *membership* (gris). Dans l'application 2D, chaque plan est affiché séparément, la navigation s'effectue par clic sur les nœuds.

D'autres algorithmes sont prévus pour les versions ultérieures et sont listés à la fin de cette section.

4.1 Détection de silos par modularité (algorithme de Louvain)

Question organisationnelle. L'organisation est-elle cloisonnée en silos qui freinent la collaboration transverse ?

Principe. L'algorithme de Louvain partitionne le graphe en communautés en maximisant la *modularité* Q, mesure définie par :

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)$$

où A_{ij} est la matrice d'adjacence (pondérée), k_i le degré du nœud i, m la somme des poids des arêtes, et $\delta(c_i, c_j) = 1$ si i et j appartiennent à la même communauté.

Interprétation. Si la partition algorithmique coïncide avec la partition formelle, l'organisation est **silotée**. Si Louvain identifie des communautés transverses aux divisions, c'est le signe d'une organisation *de fait* matricielle qu'il peut être pertinent de formaliser.

Cas TechNova. Louvain sur le graphe macro révèle deux communautés : {R&D, Produit} et {Commercial, Marketing, Ops}. Confirme statistiquement le silo R&D–Business.

4.2 Détection de goulots par centralité d'intermédiarité

Question organisationnelle. Quels nœuds (divisions, équipes, individus) constituent des goulots d'étranglement de l'information ?

Principe. La centralité d'intermédiarité (*betweenness centrality*) d'un nœud v est définie par :

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

où σ_{st} est le nombre de plus courts chemins de s à t, et $\sigma_{st}(v)$ le nombre de ces chemins passant par v. Un nœud à forte centralité d'intermédiarité est traversé par une grande proportion des flux ; son indisponibilité fragmente l'organisation.

Cas TechNova. L'équipe *Architectes* obtient une centralité de 0,68, le score le plus élevé du graphe méso. Risque de saturation : tout flux inter-équipes la traverse. Recommandation : redondance fonctionnelle (rôle de liaison secondaire) ou *by-pass* (coordination directe entre certaines équipes).

4.3 Identification des configurations théoriques (matching d'archétypes)

Question organisationnelle. À quelle configuration de Mintzberg (ou autre archétype théorique) l'organisation observée correspond-elle, et avec quels écarts ?

Principe. Pour chaque archétype de la bibliothèque (structure simple, bureaucratie mécaniste, professionnelle, divisionnelle, adhocratie), on calcule une *signature structurelle* : profondeur hiérarchique, span of control moyen, densité, modularité, ratio d'encadrement. La signature observée est comparée par distance euclidienne pondérée à chaque archétype :

$$d(\mathcal{O}, A_k) = \sqrt{\sum_i w_i (m_i^{\mathcal{O}} - m_i^{A_k})^2}$$

L'archétype le plus proche est identifié et les écarts dimension par dimension restitués à l'utilisateur. C'est l'algorithme **central** qui rend opérationnel le corpus théorique et distingue OrgaLego des outils d'analyse de réseau classiques.

Cas TechNova. La signature macro se rapproche d'une *bureaucratie professionnelle* (forte spécialisation, hiérarchie modérée, ajustement mutuel). Au niveau Backend, la signature correspond davantage à une *adhocratie*. Cette incohérence inter-niveaux est restituée comme un signal de tension nécessitant arbitrage.

4.4 Algorithmes prévus pour les versions ultérieures

Les trois algorithmes ci-dessus constituent le socle MVP. Deux algorithmes complémentaires sont identifiés pour plus tard :

— **PageRank — leaders informels** (v2). Appliqué au graphe micro de collaboration, révèlerait les acteurs réellement influents indépendamment de la hiérarchie formelle. Reporté car il suppose une qualité de données sur la collaboration informelle que les utilisateurs n'auront pas toujours.

— **Recuit simulé — optimisation automatique** (v3). Prendrait une fonction objectif pondérée (efficience, cohésion, conformité) et générerait automatiquement une structure cible optimale. Reporté car il suppose des choix méthodologiques lourds et n'est pas indispensable pour les usages pédagogiques du MVP.

L'architecture modulaire (`core/algorithms/`) permet d'ajouter ces extensions sans modifier la couche d'orchestration ni les schémas d'API.

5 Architecture technique

5.1 Vue d'ensemble

L'application repose sur une architecture client-serveur : la logique métier reste en Python (bibliothèques de référence en théorie des graphes), et le rendu interactif 2D est délégué au navigateur. Les deux couches communiquent par WebSocket pour un retour visuel en temps réel.



FIGURE 5 – Architecture générale d'OrgaLego.

5.2 Structure du projet et organisation du code

Le code est séparé en deux applications indépendantes (`backend/` en Python et `frontend/` en React), pouvant être lancées séparément. Le backend peut être développé et testé de manière autonome avant même que le frontend ne soit fonctionnel.

```
orgalego/
|
|-- backend/                          # === Application Python ===
|   |-- app/
|   |   |-- main.py                   # Point d'entree FastAPI (uvicorn)
|   |   |-- config.py                 # Settings (DB URL, CORS, env vars)
|   |   |
|   |   |-- api/                      # Endpoints exposes au frontend
|   |   |   |-- routes_blocks.py      # GET /blocks -> catalogue Lego
|   |   |   |-- routes_projects.py    # CRUD projets utilisateur
|   |   |   |-- routes_export.py      # POST /export -> PDF/Excel
|   |   |   `-- websocket_graph.py    # WS /ws/graph -> temps reel
|   |   |
|   |   |-- schemas/                  # Validation Pydantic
|   |   |   |-- block.py              # Schema d'un bloc Lego
|   |   |   |-- graph.py              # Schema graphe 3 niveaux
|   |   |   |-- diagnosis.py          # Resultats algos
|   |   |   `-- project.py            # Projet sauvegarde
|   |   |
|   |   |-- core/                     # Logique metier (coeur)
|   |   |   |-- graph_builder.py      # JSON -> NetworkX
|   |   |   |-- multilevel.py         # Gestion hierarchie 3 niveaux
|   |   |   |-- diagnosis.py          # Orchestre tous les algos
|   |   |   |
|   |   |   `-- algorithms/
|   |   |       |-- communities.py    # Louvain (silos) - MVP
|   |   |       |-- centrality.py     # Betweenness - MVP / PageRank - v2
|   |   |       |-- archetypes.py     # Matching Mintzberg - MVP
|   |   |       `-- optimization.py   # Recuit simule - v2
|   |   |
|   |   |-- blocks/                   # Bibliotheque Lego (JSON statiques)
|   |   |   |-- level1_macro/
|   |   |   |   |-- fonctionnelle.json
|   |   |   |   |-- divisionnelle.json
|   |   |   |   |-- matricielle.json
|   |   |   |   `-- adhocratie.json
|   |   |   |-- level2_meso/
```

```
|   |   |   |   |-- squad_agile.json
|   |   |   |   |-- comite_pilotage.json
|   |   |   |   `-- equipe_projet.json
|   |   |   `-- level3_micro/
|   |   |       |-- manager.json
|   |   |       |-- expert.json
|   |   |       `-- liaison.json
|   |   |
|   |   |-- db/                         # Persistance
|   |   |   |-- models.py               # Tables SQLAlchemy
|   |   |   `-- session.py              # Session DB
|   |   |
|   |   `-- services/                   # Services applicatifs
|   |       |-- block_loader.py         # Charge les .json au demarrage
|   |       `-- export_service.py       # Genere rapports PDF
|   |
|   |-- tests/
|   |   |-- test_algorithms.py
|   |   |-- test_schemas.py
|   |   `-- test_api.py
|   |
|   |-- requirements.txt
|   `-- pyproject.toml
|
|-- frontend/                           # === Application React (2D) ===
|   |-- public/
|   |   `-- index.html
|   |-- src/
|   |   |-- main.jsx                    # Point d'entree React
|   |   |-- App.jsx                     # Composant racine, layout general
|   |   |
|   |   |-- components/
|   |   |   |-- GraphView/              # Affichage du graphe 2D courant
|   |   |   |   |-- GraphView.jsx       # Conteneur React Flow / Cytoscape
|   |   |   |   |-- DivisionNode.jsx    # Bloc 2D niveau macro
|   |   |   |   |-- TeamNode.jsx        # Bloc 2D niveau meso
|   |   |   |   |-- PersonNode.jsx      # Bloc 2D niveau micro
|   |   |   |   `-- Edge.jsx            # Arete intra-niveau
|   |   |   |
|   |   |   |-- Breadcrumb/             # Fil d'Ariane (navigation par clic)
|   |   |   |   `-- Breadcrumb.jsx      # macro > R&D > Backend
|   |   |   |
|   |   |   |-- BlockPalette/           # Panneau gauche (Lego a glisser)
|   |   |   |   |-- BlockPalette.jsx
|   |   |   |   `-- BlockCard.jsx
|   |   |   |
|   |   |   |-- DiagnosisPanel/         # Panneau droit (resultats)
|   |   |   |   |-- DiagnosisPanel.jsx
|   |   |   |   |-- MetricCard.jsx
|   |   |   |   `-- RecommendationList.jsx
|   |   |   |
|   |   |   `-- Toolbar/                # Barre du haut (save, export)
|   |   |       `-- Toolbar.jsx
|   |   |
|   |   |-- hooks/                      # React hooks reutilisables
|   |   |   |-- useOrgaLegoSocket.js    # Connexion WebSocket
|   |   |   |-- useGraphStore.js        # Acces au state global
|   |   |   |-- useNavigation.js        # Gestion de la descente/remontee
|   |   |   `-- useBlockLibrary.js      # Charge le catalogue
|   |   |
|   |   |-- store/
|   |   |   |-- graphStore.js           # State global Zustand (graphe)
|   |   |   `-- navigationStore.js      # State du chemin (macro/meso/micro)
|   |   |
|   |   |-- services/
|   |   |   |-- api.js                  # Client REST (fetch wrappe)
|   |   |   `-- websocket.js            # Client WebSocket
|   |   |
|   |   `-- utils/
|   |       |-- layout2d.js             # Positions 2D des noeuds
|   |       `-- colors.js               # Metrique -> couleur
|   |
|   |-- package.json
```

```
|   `-- vite.config.js
|
|-- shared/                          # Exemples partages
|   `-- examples/
|       |-- block_example.json
|       `-- graph_technova.json      # Le cas TechNova du document
|
|-- docs/
|   |-- architecture.md
|   |-- projet_organisation.tex
|   `-- projet_organisation.pdf
|
|-- docker-compose.yml               # Lance back + front ensemble
|-- README.md
`-- .gitignore
```

Principes de cette organisation.

1. **Séparation stricte backend/frontend** : communication uniquement via l'API typée définie dans `schemas/`.
2. **Bibliothèque Lego externalisée** : `blocks/` ne contient que des JSON statiques — enrichissable sans toucher au code.
3. **Algorithmes isolés et testables** : chaque algorithme expose des fonctions pures testables indépendamment.
4. **Persistance optionnelle** : `db/` n'est requis que pour la sauvegarde de projets. L'application fonctionne sans BDD en mode démo.

5.3 Backend Python : bibliothèques et rôle de chacune

Bibliothèque	Catégorie	Rôle dans le projet
`NetworkX`	Graphes	Bibliothèque principale : représentation des graphes, algorithmes (centralités, plus courts chemins, partitionnement).
`python-louvain`	Communautés	Implémentation de référence de l'algorithme de Louvain pour la détection de silos.
`numpy / scipy`	Calcul scientifique	Algèbre linéaire (matrices d'adjacence, calculs spectraux), optimisation numérique (prévu pour v2).
`FastAPI`	API web	Framework asynchrone pour exposer les fonctions Python en API REST et WebSocket.
`uvicorn`	Serveur ASGI	Serveur de production pour FastAPI.
`Pydantic`	Validation	Schémas typés pour valider les blocs Lego, les structures soumises, les requêtes.
`SQLAlchemy`	ORM	Persistance des projets utilisateurs (SQLite en local, PostgreSQL en SaaS).
`pandas`	Données	Import/export Excel et CSV pour les utilisateurs qui partent d'un organigramme existant.
`matplotlib`	Visualisation	Génération d'exports PDF des diagnostics (rapports).

TABLE 2 – Stack Python d'OrgaLego.

Rôle des modules clés.

— `api/` : porte d'entrée. Reçoit les requêtes HTTP/WebSocket, valide avec Pydantic, délègue à `core/`.

— `schemas/` : « grammaire » partagée avec le frontend (bloc, graphe, diagnostic). Garantit que les deux côtés parlent la même langue.

— `core/graph_builder.py` : convertit le JSON envoyé par le frontend en objet NetworkX manipulable.

— `core/algorithms/` : un fichier par algorithme. Fonctions pures testables.

— `core/diagnosis.py` : orchestrateur. Appelle Louvain, betweenness et matching d'archétypes, agrège les résultats. Modulaire pour accueillir PageRank et recuit simulé (v2).

— `blocks/*.json` : bibliothèque Lego, externalisée pour pouvoir l'enrichir sans toucher au code.

5.4 Frontend : choix techniques

Le frontend utilise **React** comme base, avec une bibliothèque dédiée au rendu de graphes interactifs en 2D :

— **React Flow** ou **Cytoscape.js** : bibliothèques 2D pour afficher des nœuds et arêtes interactifs (drag-and-drop, sélection, zoom, mise en page automatique).

— **Zustand** : gestionnaire d'état global léger qui maintient l'état du graphe en cours d'édition et déclenche les envois WebSocket.

L'utilisateur navigue entre les trois niveaux du graphe hiérarchique par **clic** : un clic sur un nœud du niveau supérieur ouvre la vue du sous-graphe associé au niveau inférieur. Un fil d'Ariane en haut de l'écran permet de remonter à tout moment.

Méthodologie de développement du frontend. L'intégralité du code React sera développée avec l'assistance d'un grand modèle de langage (LLM) :

— L'effort intellectuel principal porte sur la **modélisation organisationnelle** et les **algorithmes**, écrits manuellement en Python.

— Le code frontend (composants, glisser-déposer, hooks WebSocket) relève de patterns bien couverts par les LLM modernes.

— Cette répartition concentre le temps sur la partie scientifique tout en livrant rapidement une interface fonctionnelle.

Les schémas Pydantic du backend servent de contrat d'interface stable pour guider la génération du code par le LLM.

5.5 Communication backend ↔ frontend

Le canal retenu est le **WebSocket**, choisi pour sa bidirectionnalité (le serveur peut pousser des mises à jour), sa faible latence (idéal pour le retour temps réel) et sa persistance de connexion (état de session maintenu côté serveur).

Protocole applicatif. Messages JSON typés par Pydantic, trois catégories :

— `graph_update` : delta de modification envoyé par le frontend.

— `diagnosis_result` : métriques et recommandations renvoyées par le backend.

— `optimization_request` / `optimization_progress` : prévus pour v2/v3 (recuit simulé).

Flux de données. La figure 6 retrace le cycle complet d'une interaction utilisateur : du clic ou du glisser-déposer jusqu'à la mise à jour visuelle reflétant le diagnostic recalculé.

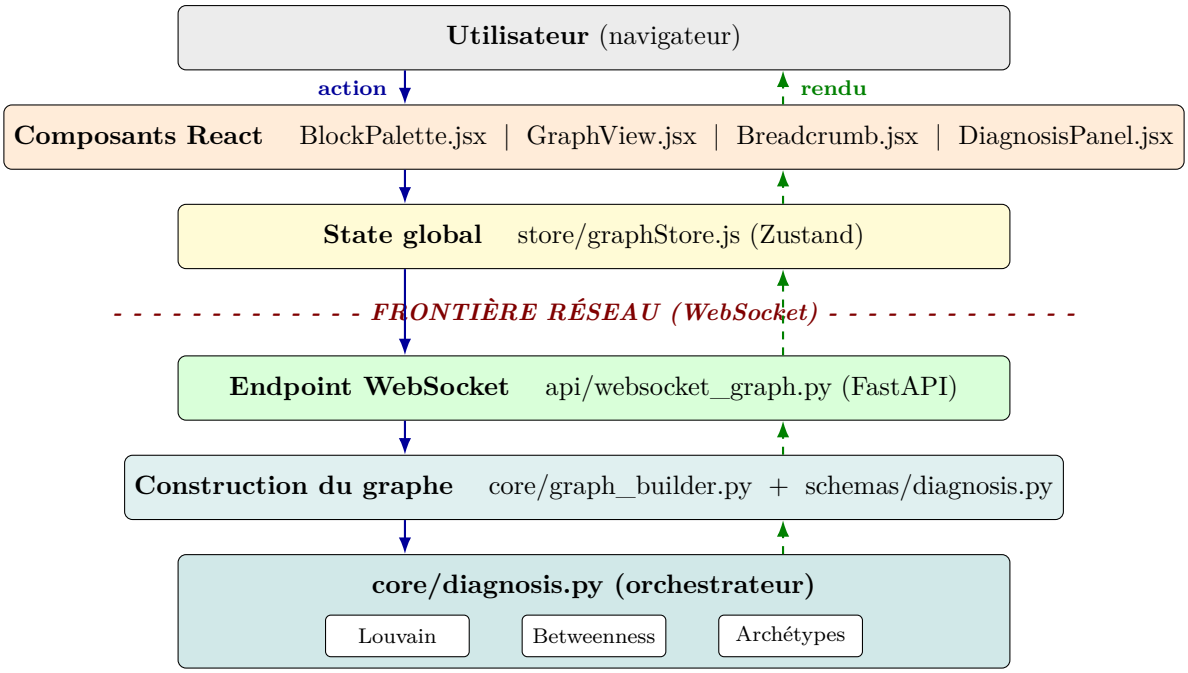


FIGURE 6 – Flux de données entre frontend et backend lors d'une interaction utilisateur. Les flèches bleues (descendantes) acheminent une modification du graphe depuis le navigateur jusqu'aux algorithmes ; les flèches vertes pointillées (montantes) ramènent les résultats du diagnostic pour mise à jour de l'interface 2D. L'ensemble du cycle se déroule typiquement en moins de 100 ms, donnant l'impression d'un calcul en temps réel.

Synthèse en une phrase. Le frontend décrit *l'état visuel* du graphe, le backend décrit *ce que cet état signifie* (métriques, diagnostics, recommandations), et le WebSocket les fait dialoguer en quasi temps réel.

6 Synthèse du scénario TechNova

Pour boucler l'illustration, voici comment OrgaLego accompagne TechNova de son diagnostic à sa restructuration optimisée.

Étape 1 — Modélisation initiale. L'utilisateur importe l'organigramme existant ou le reconstruit en assemblant des blocs de la bibliothèque. La structure macro (figure 1) est identifiée comme *fonctionnelle pure*.
Étape 2 — Diagnostic automatique. L'application calcule en temps réel les trois métriques fondamentales du MVP : modularité de Louvain (silo R&D/Business confirmé), centralités d'intermédiarité (équipe Architectes saturée), et distance aux archétypes (bureaucratie professionnelle au niveau R&D, incohérente avec l'adhocratie observée au niveau Backend).
Étape 3 — Hypothèses de réorganisation. L'utilisateur teste plusieurs configurations en glissant-déposant des blocs : (a) ajout d'un rôle de *Product Manager* de liaison entre R&D et Commercial ; (b) bascule en structure matricielle ; (c) création de *squads* pluridisciplinaires. Pour chaque hypothèse, les trois algorithmes sont recalculés et restituent leur diagnostic instantanément.
Étape 4 — Arbitrage et restitution. L'utilisateur compare les configurations testées sur les indicateurs clés (modularité, centralité maximale, distance à l'archétype cible) et retient celle qui réalise le meilleur compromis selon ses priorités. Un rapport PDF est généré, comparant la structure initiale et la structure cible sur l'ensemble des métriques, avec justification théorique des choix.

7 Conclusion et perspectives

OrgaLego propose une approche *theory-driven* et *learning-oriented* de la conception organisationnelle, complémentaire des outils enterprise existants centrés sur la donnée HR. La modélisation en graphe hiérarchique à trois niveaux est mathématiquement tractable, pédagogiquement lisible, et ouverte à l'enrichissement progressif.

Périmètre du MVP. La première version livrée sera entièrement **en 2D** avec navigation par clic entre les trois niveaux, une bibliothèque initiale de 6 à 8 blocs Lego couvrant les configurations de Mintzberg, et les trois algorithmes essentiels (Louvain, centralité d'intermédiarité, matching d'archétypes). Ce périmètre est suffisant pour produire un diagnostic organisationnel complet et défendable, et pour valider l'outil auprès de premiers utilisateurs étudiants ou formateurs.

Si le temps le permet : extensions prévues. Une fois le MVP 2D stabilisé et validé, deux extensions sont envisagées dans l'ordre suivant :

1. **Algorithmes d'optimisation supplémentaires** : implémentation de PageRank pour identifier les leaders informels au niveau micro, puis du recuit simulé pour générer automatiquement des structures cibles optimisées selon une fonction objectif pondérée (efficience, cohésion, conformité à un archétype).
2. **Passage en 3D** : ajout d'une vue tridimensionnelle alternative dans laquelle les trois niveaux apparaissent simultanément, empilés et reliés par les arêtes verticales de *membership* (cf. figure 4). L'utilisateur pourra alors basculer entre la vue 2D par niveau (mode d'édition principal) et la vue 3D synthétique (mode d'exploration). Cette étape s'appuiera sur React Three Fiber, Three.js et `@react-three/drei`.

L'architecture du projet (séparation backend/frontend, schémas Pydantic comme contrat d'interface, fichier dédié par algorithme) est conçue pour absorber ces évolutions sans réécriture profonde.